

Machine Learning

A new kind of Science

Machine Learning: Definition

- Arthur Samuel (1959): *“The field of study that gives computers the ability to learn without being explicitly programmed.”*
- Tom Mitchell (1998): *“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .“*



T: Playing checkers

P: Percentage of games won against an arbitrary opponent

E: Playing practice games against itself

Example: Learning to score reviews



Score = -1.5

“Slow moving. Couldn’t get into this movie despite all the awards it has won.”

Score = 2.5

“Wow! Did not know what to expect and was delighted! Loved the homage paid to the musicals of old. “

Given: Reviews for different movies and rating score (-3 to +3).

Task: Learn to **predict** rating score given text of a **new** review.



The “law” of movie review scores??

Example: Learning to rate reviews (contd)

“Slow moving. Couldn’t get into this movie despite all the awards it has won.”

100,000 words in English dictionary.

Hypothesized “law” (simple linear model):

- Each word w has sentiment score $\theta_w \in \mathbb{R}$

Review score $\approx$ total sentiment score of all words in it

Finding sentiment score for 100,000 words $\equiv$ Surface fitting with 100,000 variables!

R¹⁰⁰⁰⁰

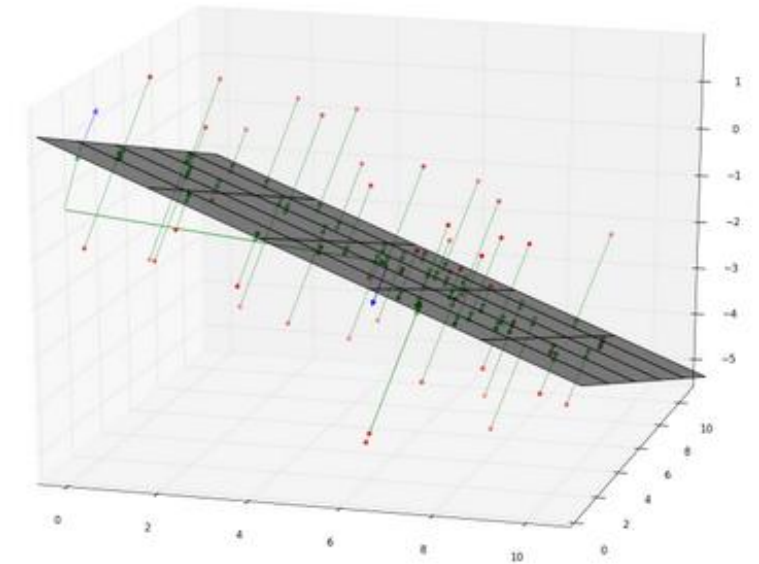
100,000 words in English dictionary.

Hypothesized “law” (simple linear model):

- Each word w has sentiment score θ_w
Review rating $\approx$ total sentiment score of all words in it

$$\text{Min} \sum_{\text{review}} \left(\text{Score} - \sum_{w \in \text{review}} \theta_w \right)^2$$

“Loss function”



Algorithm: “**Gaussian Least Square Fit**”;
(Solvable in seconds given few million ranked reviews.)

Hypothesized law:

- Each word w has sentiment score θ_w

Review rating $\approx$ total sentiment score of all words in it

Word	Score
as	+0.1
good	+0.6
homage	0.0
loved	+1.1
musicals	+0.1
of	-0.1
old	-0.4
paid	-0.3
story	+0.3
the	+0.1
to	-0.1
was	-0.2
well	+1.4
⋮	⋮

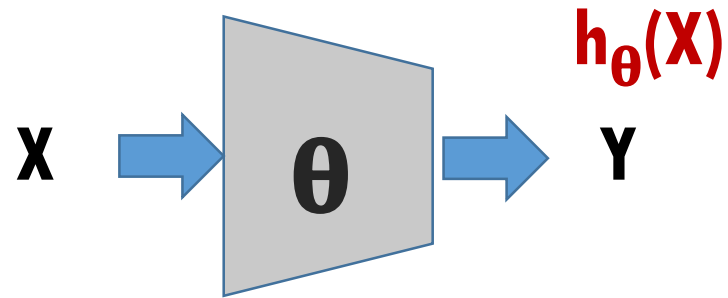
“Loved the homage paid to the musicals of old. Story was good as well.”

Loved the homage paid to the musicals of old. Story was good as well.
+1.1 +0.1 +0.0 -0.3 -0.1 +0.1 +0.1 -0.1 -0.4 +0.3 -0.2 +0.6 +0.1 +1.4 = 2.7

“Discovered Patterns” =
How words correlate with
positive score



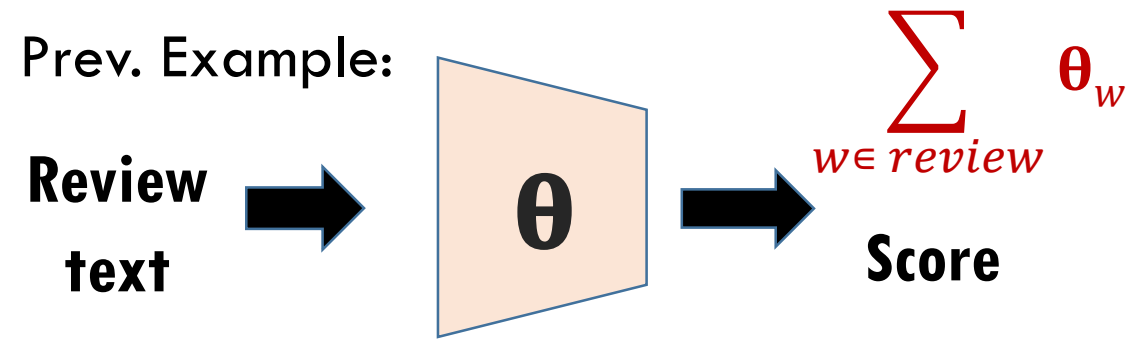
ML $\approx$ finding suitable function (“model”) given examples of desired input/output behavior



$$\theta \in \mathbb{R}^d$$

(trainable parameters)

(X, Y) : (Input, Output) pair



Training method:

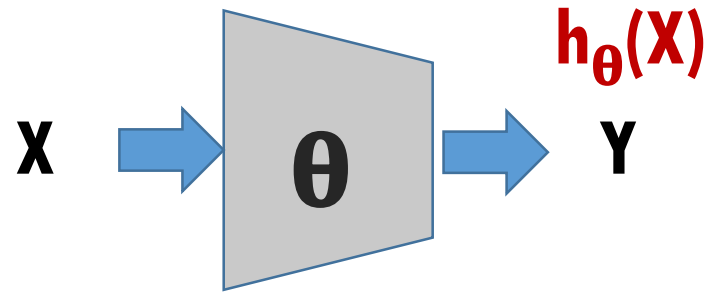
2

$$\text{Min} \sum_{\text{review}} \left(\text{Score} - \sum_{w \in \text{review}} \theta_w \right)^2$$

Desired output

Model's output

ML $\approx$ finding suitable function (“model”) given examples of desired input/output behavior



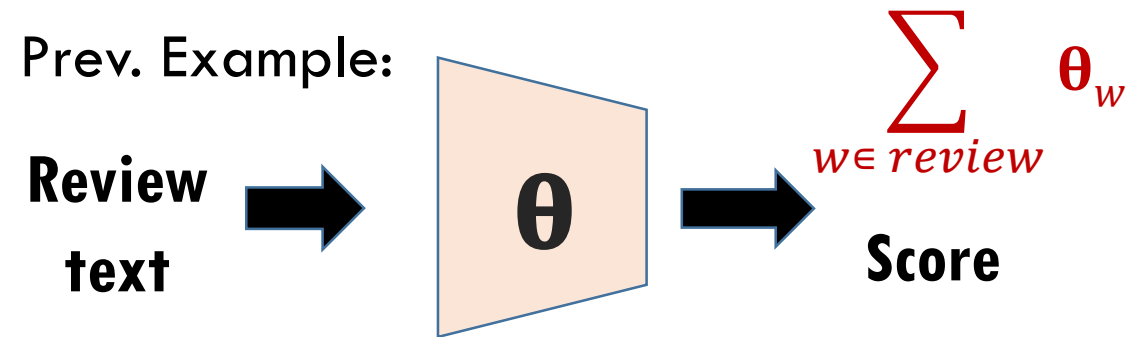
$$\theta \in \mathbb{R}^d$$

(trainable parameters)

Training data: $\{(X^{(i)}, Y^{(i)}): i=1, \dots, n\}$

“Loss” $J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$

(Many other loss formulations exist....)



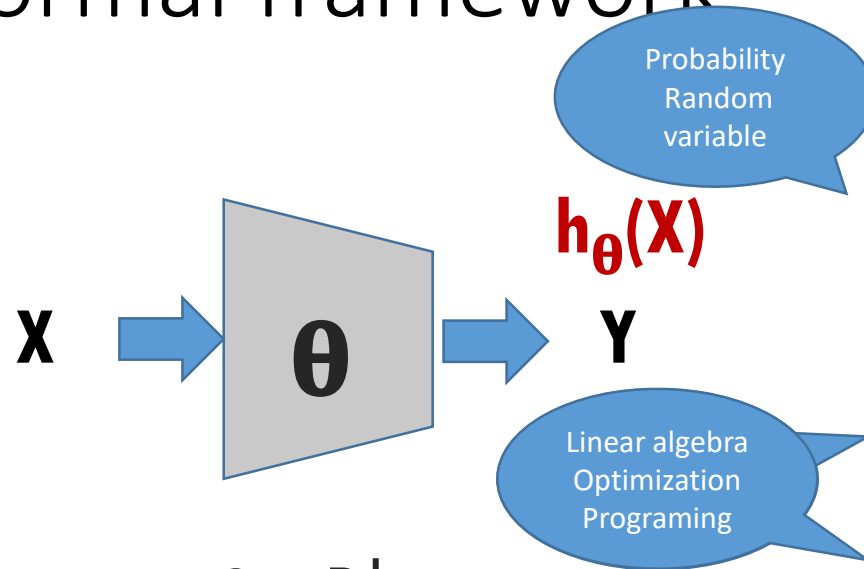
Training method: 2

$$\text{Min} \sum_{\text{review}} \left(\text{Score} - \sum_{w \in \text{review}} \theta_w \right)^2$$

Desired output Model's output

Formal framework

(inherited from classical statistics)



$\theta \in \mathbb{R}^d$

(trainable parameters)

Training data: $\{(X^{(i)}, Y^{(i)}): i=1, \dots, n\}$

“Loss”
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

(Many other loss formulations exist....)

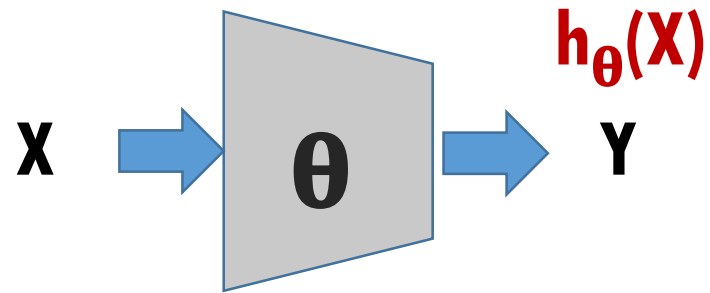
Assumption: Samples $(X^{(i)}, Y^{(i)})$ drawn from a **probability distribution D** (e.g., distribution on pairs “(movie review, rating score)”)

TRAINING: Optimize loss $J(\theta)$.

TESTING: Take **new** X ’s and see how well the trained model **predicts the missing Y** (e.g., “given review, predict rating score”)

In practice: TRAIN on 80% of data;
TEST by evaluating loss on remaining 20%

Formal framework



$\theta \in \mathbb{R}^d$
(trainable parameters)

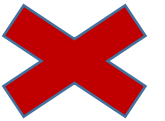
Training data: $\{(X^{(i)}, Y^{(i)}): i=1, \dots, n\}$

“Loss”
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

(Many other loss formulations exist....)

Assumption: Samples $(X^{(i)}, Y^{(i)})$ drawn from a probability distribution D (e.g., distribution on pairs “(movie review, rating score)”)

Comment: Fourier analysis allows learning f from $(x, f(x))$ examples.....
(under reasonable assumptions on f)....

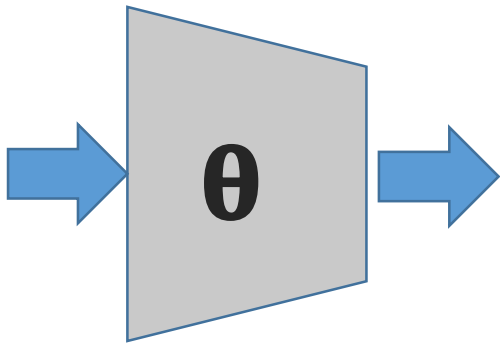


But practically infeasible: If $X \in \mathbb{R}^n$, need $\exp(n)$ samples of $(X, f(X))$ and also $\exp(n)$ computation time.

(In practice $n = 10^4$ say, and # samples = $10n$)

Training via Gradient Descent (“natural algorithm”)

$\theta \in \mathbb{R}^d$
trainable parameters



Loss $J(\theta)$

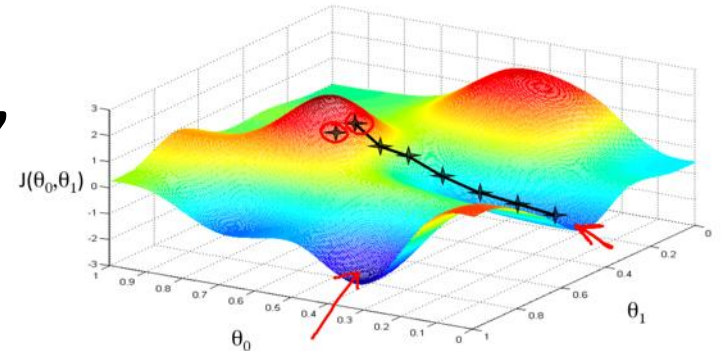
deficiency in desired fit

**Often nonconvex,
esp. in deep
learning.**

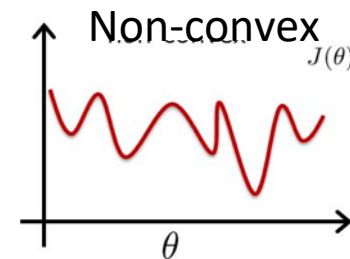
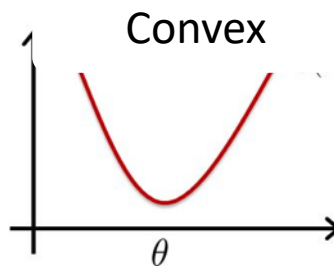
Starting with some $\theta^{(0)}$,
compute for $t=0,1,2,\dots$

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla (J)$$

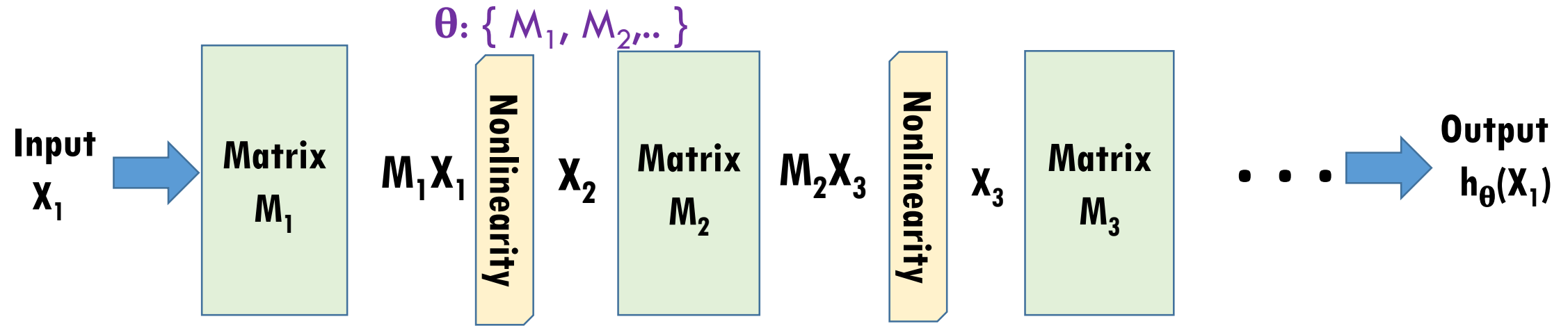
($\alpha \in \mathbb{R}$ is “learning rate”, say, 0.01)



In practice can improve GD with tricks: time-varying α , past gradients (“momentum”), “regularization”, ..



Subcase: deep learning* (deep models = “multilayered”)



“Nonlinearity”: Given a vector, output same vector but negative entries turned to zero.

Training data: $\{(X_1^{(i)}, Y^{(i)}): i=1, \dots, n\}$

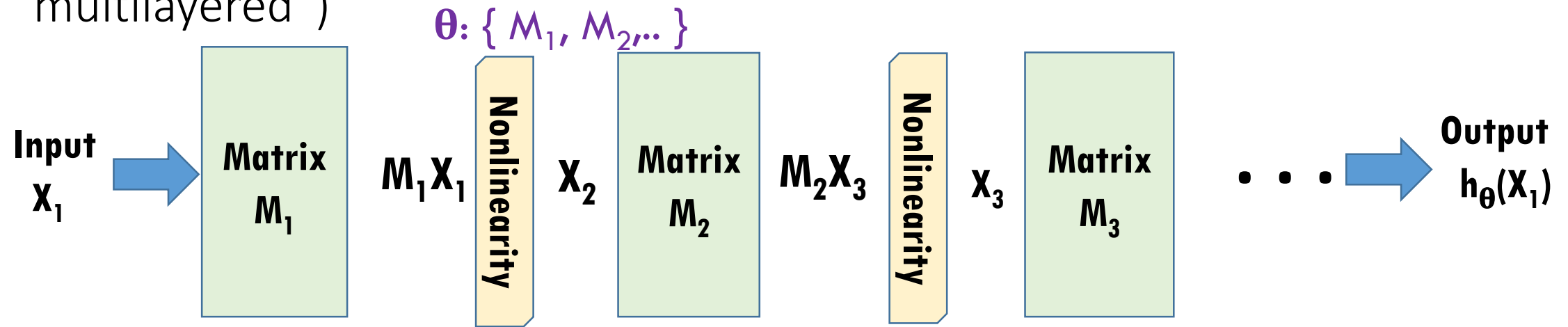
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_\theta(x^{(i)}) - y^{(i)})^2$$

How to compute gradient of Loss??

Backpropagation Algorithm does it fast; clever application of **chain rule** [Werbos'77, Rumelhart et al'84]

(*Highly simplistic: could have “convolution”, “bias”, “skip connections”, other loss fns etc.)

Subcase: deep learning* (deep models = “multilayered”)



“Nonlinearity”: Given a vector, output same vector but negative entries turned to zero.

Training data: $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$

How to compute

Backpropagation

Nonlinear enough to express many things; linear enough to allow quick optimization on today's computers

[LeCun, Bengio, Hinton, Rumelhart et al'84]

(*Highly simplistic: could have “convolution, bias, skip connections”, other loss fns etc.)



Review rating = total sentiment score of all words in it $\sum_{w \in \text{review}} \theta_w$

This is not how we humans do it!

(i) Word order in text matters. (“No good” vs “Good, no?”)

*(ii) Don’t need millions of examples. (Because we **understand** language!)*

Science has long tried to understand and formalize language + semantics
(using logic, grammars, model theory,...)

How can machines “understand” language?

(Arguably, more general-purpose than rating reviews!)

Unsupervised learning (no human-supplied labels)

- Key idea: Using large corpus (eg, Wikipedia), train model to predict **part of text** from **adjacent** text.

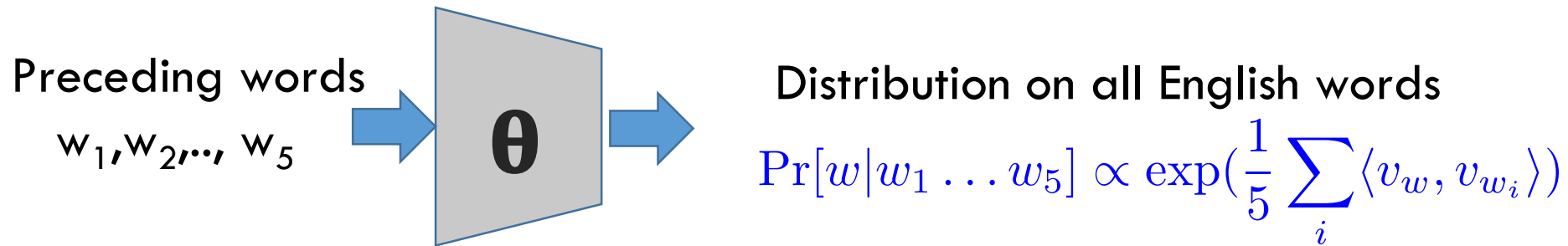
Example: I went to a tea stall and ordered a....

Model learns to do such completions by training on huge amounts of text....

(In the process, **implicitly** picks up on grammar rules, common sense etc.)

Used in: Machine Translation, Question-Answering (e.g. Siri, Alexa..)

A Language model (baby “word2vec” [mikolov et al’13])



$$\boldsymbol{\theta} = \{ \mathbf{v}_w \in \mathbb{R}^{300} : w \text{ an English word} \}$$

(“semantic vectors”)

Loss $\ell(\boldsymbol{\theta})$: **Reciprocal** of Probability assigned
by model to Wikipedia = $w_1 w_2 w_3 \dots w_N$

$$\prod_{i=6}^N \Pr[w_i | w_{i-5}, \dots, w_{i-1}]$$
$$= \exp\left(\sum_{i=6}^N \sum_{j=1}^5 \frac{1}{5} \langle v_{w_i}, v_{w_{i-j}} \rangle\right)$$